Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Représentation binaire des entiers relatifs, arithmétique bas niveau et détection du dépassement de capacité

Synthèse exécutive : Ce cours détaille les mécanismes matériels et algorithmiques utilisés par les processeurs pour représenter et manipuler les nombres entiers relatifs (positifs et négatifs) en mémoire binaire sur une largeur fixe de N bits. En raison de l'impossibilité physique des composants électroniques (transistors, bascules bistables) de stocker directement un symbole abstrait '+' ou '-', l'architecture informatique moderne s'appuie universellement sur la convention du complément à deux. Cette méthode élimine le problème du double zéro (+0 et -0) présent dans la représentation Signe et Magnitude et unifie l'addition et la soustraction au sein d'un circuit additionneur unique dans l'Unité Arithmétique et Logique (ALU). Le cours explicite l'algorithme de conversion en trois étapes, le calcul des plages de valeurs dissymétriques [-2^(N-1), +2^(N-1)-1], la gestion de la retenue sortante (Carry Out), ainsi que les règles formelles de détection de l'Overflow (dépassement de capacité).

Problématique matérielle et justification de la convention du Complément à Deux

Au niveau le plus élémentaire de la hiérarchie matérielle d'un processeur, l'information est stockée dans des cellules mémoire et des registres constitués de transistors. Ces circuits bistables fonctionnent de manière binaire : soit le courant ne passe pas (état 0, 0 Volt), soit le courant passe (état 1, tension Vcc). En conséquence, le processeur ne dispose d'aucun mécanisme physique direct pour représenter le caractère '+' ou '-'. Pour encoder des entiers négatifs, la première approche théorique a été la représentation 'Signe et Magnitude'. Cependant, elle présente deux inconvénients majeurs en architecture informatique : d'une part, elle génère deux motifs binaires distincts pour représenter la valeur zéro (+0 et -0), ce qui implique une perte d'efficacité mémoire et oblige les circuits de comparaison à effectuer des tests supplémentaires ; d'autre part, elle contraint les concepteurs de puces à fabriquer des circuits physiques séparés pour les additions et les soustractions. Pour surmonter ces contraintes, l'architecture des ordinateurs utilise la convention du complément à deux sur N bits. Dans ce système, le zéro possède une signature unique (tous les bits à 0) et l'opération de soustraction A - B est directement calculée sous la forme A + (-B). L'Unité Arithmétique et Logique (ALU) n'a donc plus besoin d'un circuit soustracteur séparé : elle utilise le même additionneur binaire pour traiter indifféremment les additions et les soustractions.

- Absence de signe matériel : Les composants électroniques (transistors, bascules bistables) ne stockent que deux états logiques : 0 (0 Volt) ou 1 (Vcc). Aucun composant ne peut enregistrer un signe '+' ou '-' physique.
- Limites du codage Signe et Magnitude : Cette méthode réserve le bit de poids fort (MSB) au signe (0 = positif, 1 = négatif) et les bits restants à la valeur absolue. Elle produit un double zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison binaire et impose de concevoir deux circuits électroniques séparés dans l'ALU (un additionneur et un soustracteur).
- Unification par le Complément à Deux : Cette convention attribue au zéro une représentation binaire unique (0000...0) et transforme toute soustraction A - B en une addition A + (-B), permettant de réutiliser le même additionneur binaire au niveau de l'ALU.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur est bloqué à 0000 km et que l'on effectue un kilomètre en marche arrière, le compteur repasse mécaniquement à 9999. En binaire sur 8 bits, soustraire 1 à 00000000_2 donne 11111111_2, ce qui constitue la définition formelle du nombre -1 en complément à deux.*

Algorithme de Conversion et Plage de Valeurs sur N bits

La représentation d'un nombre négatif -X en complément à deux nécessite obligatoirement de définir au préalable la taille N du mot mémoire (ex: 8, 16, 32 ou 64 bits). L'algorithme se déroule selon une procédure méthodique en trois étapes indissociables. La première étape consiste à convertir la valeur absolue X en binaire pur sur N bits. Il

est fondamental de rajouter tous les zéros nécessaires sur la gauche pour occuper la totalité des N bits de l'architecture. La deuxième étape consiste à appliquer le complément à un, c'est-à-dire permuter tous les bits du mot (remplacer les 0 par des 1 et les 1 par des 0). La troisième étape consiste à ajouter la valeur 1 au nombre binaire obtenu pour former le complément à deux. Dans cette représentation, le bit le plus à gauche (MSB pour Most Significant Bit) joue le rôle d'indicateur de signe : une valeur de 0 indique un nombre positif ou nul, tandis qu'une valeur de 1 atteste que le nombre est négatif. La plage des valeurs représentables sur N bits est dissymétrique. Cette dissymétrie provient du fait que la combinaison zéro (0000...0) possède un MSB égal à 0 et est donc rattachée à la plage des nombres positifs. Ainsi, sur 8 bits, il est possible de représenter 256 valeurs différentes allant de -128 à +127 (et non pas de -127 à +127).

- Algorithme séquentiel des 3 étapes : Pour coder un nombre négatif -X sur N bits : 1) Écrire X en binaire naturel sur N bits complets (cadrage à gauche avec des 0) ; 2) Inverser chaque bit (Complément à 1) ; 3) Ajouter 1 au résultat binaire (Complément à 2).
- Rôle du bit MSB (Most Significant Bit) : Le bit de poids fort (le plus à gauche) sert d'indicateur de signe (0 = positif ou nul, 1 = négatif).
- Dissymétrie de la plage de valeurs : Sur un mot de N bits, les valeurs représentables s'étendent de -2^(N-1) jusqu'à +(2^(N-1) - 1). Sur 8 bits, cela correspond à l'intervalle [-128, +127].

Cas concret enseignant : *Avertissement crucial pour les contrôles et partiels : il faut IMPÉRATIVEMENT fixer la taille N bits (ex: 8 bits) et écrire les zéros de cadrage à gauche AVANT d'effectuer l'inversion des bits. Inverser la séquence '1100' (+12 sur 4 bits) sans ajouter au préalable les 4 zéros de cadrage '00001100' donne '0011' (+3 décimal) au lieu de '11110011' (-12 décimal sur 8 bits), ce qui conduit à une réponse totalement fausse en examen.*

Arithmétique Binaire, Retenue (Carry Out) et Détection du Dépassement de Capacité (Overflow)

Lorsque le processeur exécute des calculs arithmétiques dans un registre de largeur fixe N bits, deux indicateurs d'état (flags/drapeaux) doivent être scrupuleusement distingués au niveau du registre du processeur : le Carry Out (C) et l'Overflow (OV). Le Carry Out correspond à la retenue sortante générée lors de l'addition du bit de poids fort MSB (position N+1). En arithmétique signée en complément à deux, cette retenue sortante n'indique pas une erreur calculatoire : elle est simplement ignorée car le matériel effectue ses calculs modulo 2^N. Le résultat contenu dans les N bits du registre reste parfaitement exact. À l'inverse, l'Overflow (OV) signale une erreur critique de dépassement de capacité. Il survient lorsque la valeur exacte du résultat dépasse la valeur maximale (ou minimale) codable sur N bits, ce qui altère le bit de signe (MSB). Pour détecter formellement un Overflow lors d'un examen, la règle absolue est la suivante : un Overflow se produit si et seulement si l'on additionne deux opérandes ayant le même bit de signe (soit deux nombres positifs, soit deux nombres négatifs) et que le résultat présente un bit de signe inverse. Si l'on additionne un nombre positif (MSB=0) et un nombre négatif (MSB=1), l'apparition d'un Overflow est mathématiquement et physiquement impossible.

- Soustraction unifiée : Calculée directement sous la forme A - B = A + (-B) par l'ALU.
- Gestion du Carry Out (retenue sortante) : La retenue générée au-delà du bit MSB (position N+1) est systématiquement ignorée en complément à deux car le processeur calcule en arithmétique modulaire modulo 2^N.
- Définition du drapeau d'Overflow (OV) : Indique un dépassement de capacité lorsque le résultat mathématique excède l'intervalle représentable [-2^(N-1), +2^(N-1)-1].
- Règle d'or de détection d'Overflow : Un Overflow survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé. L'addition de deux nombres de signes différents ne peut jamais générer d'Overflow.

Cas concret enseignant : *L'anecdote historique du crash de la fusée Ariane 5 (vol 501 en 1996) : une conversion de type d'un nombre flottant 64 bits vers un entier signé 16 bits sans protection contre le dépassement de capacité a provoqué un Overflow non géré. L'Unité de Référence Inertielle a transmis une valeur corrompue au calculateur de bord, interprétée à tort comme une déviation de trajectoire, ce qui a déclenché le braquage brutal des tuyères puis l'autodestruction de la fusée 37 secondes après le décollage.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant du bit MSB) est-elle systématiquement jetée lors de l'addition en complément à deux ?
Réponse du professeur : Parce que les registres du processeur ont une largeur fixe de N bits et effectuent les opérations en arithmétique modulaire (modulo 2^N). La retenue qui dépasse au bit N+1 tombe hors du registre. Mathématiquement, retrancher 2^N équivaut à conserver la valeur exacte du complément à deux sur les N bits restants.

Question étudiant : Comment l'ordinateur fait-il la différence en mémoire entre le mot binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur ne fait pas la différence physiquement au niveau du registre mémoire : la suite de bits 11111111 est strictement identique. C'est l'instruction exécutée par le processeur (ou le type de donnée déclaré par le programmeur en C/C++ comme 'unsigned char' ou 'signed char') qui détermine l'interprétation arithmétique de ce motif binaire.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler le codage binaire des deux opérandes en complément à deux, poser l'addition binaire correspondante, donner le résultat décimal final et analyser la valeur du drapeau d'Overflow (OV).

Démarche de résolution :
Étape 1 : Codage du premier opérande (+35) sur 8 bits. 35 = 32 + 2 + 1 = 00100011_2. Le bit MSB est 0 (nombre positif). Étape 2 : Codage du second opérande (-75) en complément à deux sur 8 bits. - Valeur absolue (+75) en binaire naturel sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion de tous les bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation binaire de -75 sur 8 bits). Étape 3 : Pose et calcul de l'addition binaire (+35) + (-75). 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 Étape 4 : Interprétation du résultat binaire 11011000_2. Le bit le plus à gauche (MSB) vaut 1, le résultat est donc négatif. Pour déterminer sa valeur absolue décimale, on réapplique la procédure du complément à deux sur 11011000_2 : - Inversion des bits : 00100111_2 - Ajout de 1 : 00100111 + 1 = 00101000_2 = 32 + 8 = 40 décimal. Le mot binaire 11011000_2 représente donc la valeur -40 en décimal. Étape 5 : Analyse du drapeau de dépassement de capacité (Overflow). On a additionné un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). L'addition de deux opérandes de signes opposés ne peut jamais produire d'Overflow.

Résultat : Résultat binaire final : 11011000_2 (égal à -40 en décimal). Drapeau d'Overflow OV = 0 (aucun dépassement de capacité). Le calcul 35 - 75 = -40 est exact et validé.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage des entiers signés représentables sur un mot mémoire de 16 bits en complément à deux ?

R : La plage s'étend de -2^(16-1) à +(2^(16-1) - 1), soit de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors d'une addition de deux entiers signés ?

R : Un Overflow survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé (MSB inversé).

Q : Quel est le résultat de l'application de l'algorithme du complément à deux sur le nombre zéro (00000000_2) sur 8 bits ?

R : Inversion des bits : 11111111_2. Ajout de 1 : 11111111 + 1 = 100000000_2. Sur 8 bits, le bit de retenue tout à gauche est rejeté, ce qui donne 00000000_2. Le complément à deux de zéro donne zéro, prouvant l'unicité du codage.